KYR - Know Your Repo

Turn any GitHub repository into a personalized learning experience.

- Architecture Analysis - Understanding
- Code Debugger - Problem Solving
- System Design - Optimization
- Code Extension - Improving

How KYR Works: The Complete Workflow

1. User Enters GitHub URL

User provides repository link via **React** frontend

2. Backend Fetches Repository (FastAPI)

GitHub API retrieves all files, code, and documentation

3. Code Chunking (LangChain + ChromaDB)

Repository broken into logical chunks and embedded in vector database

4. Retrieval with RAG

Relevant **chunks** retrieved based on user queries

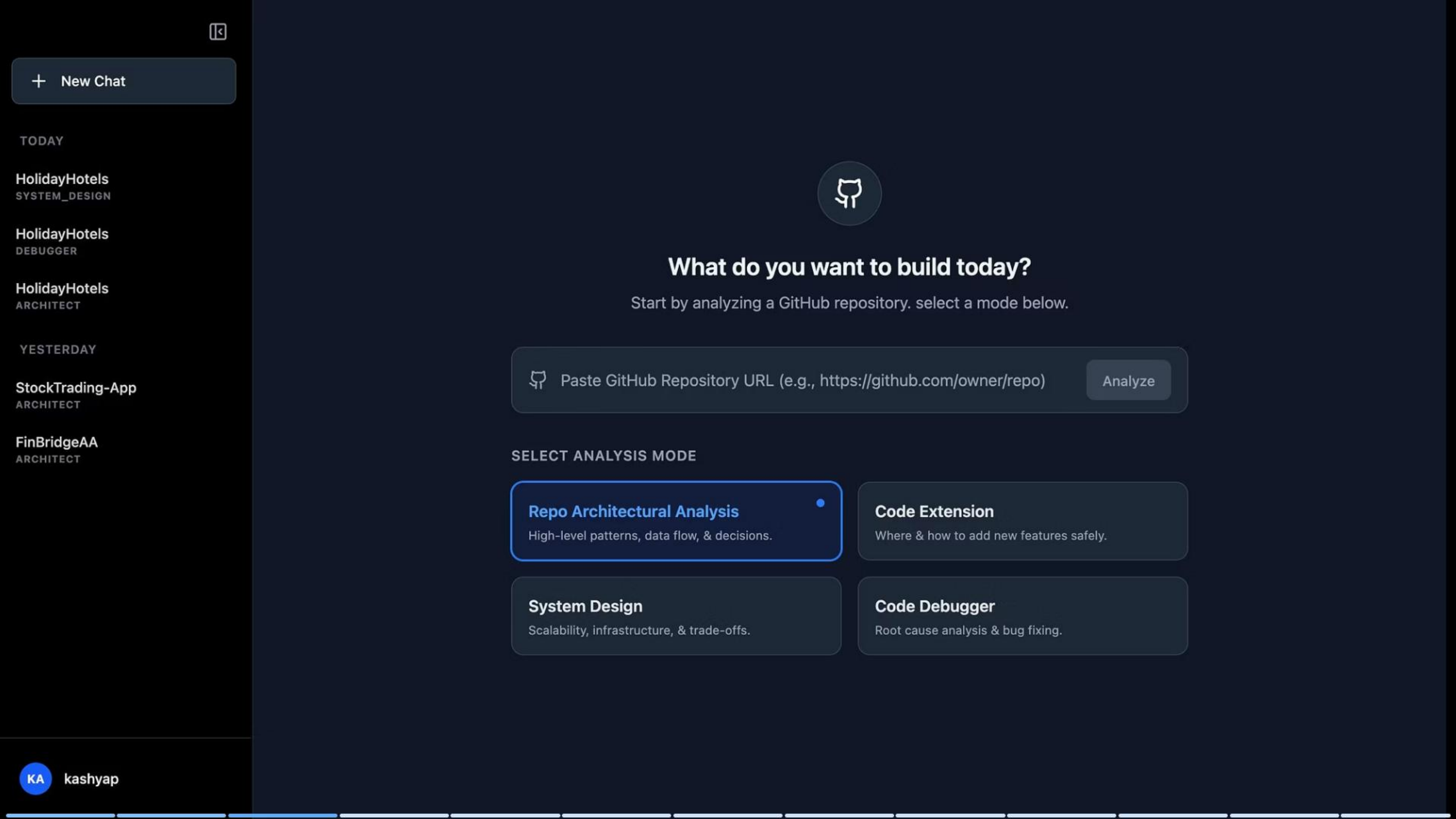
5. Gemini LLM Processes

LLM generates context-aware explanations and solutions

6. Output to User

Phase-wise learning + interactive chatbot on frontend

Full-repo understanding → Intelligent chunking → RAG retrieval → Accurate LLM responses → Interactive learning experience


New Chat
TODAY
HolidayHotels
SYSTEM_DESIGN
HolidayHotels
DEBUGGER
HolidayHotels
ARCHITECT
YESTERDAY
StockTrading-App
ARCHITECT
FinBridgeAA
ARCHITECT
KA
kashyap
What do you want to build today?
Start by analyzing a GitHub repository. select a mode below.
Paste GitHub Repository URL (e.g., https://github.com/owner/repo)
Analyze
SELECT ANALYSIS MODE
Repo Architectural Analysis
High-level patterns, data flow, & decisions.
Code Extension
Where & how to add new features safely.
System Design
Scalability, infrastructure, & trade-offs.
Code Debugger
Root cause analysis & bug fixing.

KYR's Four Core Repo Understanding Capabilities:

Architecture Analysis — Understanding

Breaks the repository into logical phases and explains how folders, files, and modules connect. Shows the overall flow of data and execution. What and Why aspects of a repo.

Outcome: Clear mental model of the entire project

System Design — Scaling

Analyzes system structure and dependencies. Identifies bottlenecks and suggests improvements for scalability and performance.

Your own Senior-Developer always with you.

Outcome: A system that's ready to grow

Code Debugger — Problem Solving

Analyzes errors and unexpected behavior across the entire repository. Guides users step-by-step to the root cause with full repo context.

Guides for maintainable and future ready code.

Outcome: Faster, more accurate debugging with confidence

Code Extension — Improving

Helps safely add or enhance features. Analyzes impact before suggesting changes and aligns new features with existing architecture.

Suggest full workflow change needed to implement a new feature.

Outcome: Cleaner, safer, future-proof feature development

Four ways to understand, improve, and scale: Understand → Solve → Scale → Improve. One platform for all your repository needs.

PHASE 3 OF 15

Phase 2: Technology Stack & Constraints

Okay, here's the technical analysis of the provided repository content, focusing on Phase 2: Technology Stack & Constraints.

- **WHAT**: This excerpt from a `package-lock.json` file describes the dependency graph of a Node.js project. It defines the versions, sources, integrity hashes, and dependencies of each installed package. It exposes which frameworks are used, and framework versions and known deprecated libraries.
- **HOW**: The `package-lock.json` file is automatically generated and updated by `npm` or `yarn` during package installation.
  - Each entry represents a specific version of an npm package. The `version` field indicates the installed version.
  - The `resolved` field points to the exact location of the package in the npm registry.
  - The `integrity` field is a cryptographic hash used to verify the package's integrity during installation.
  - The `dependencies` field lists the package's direct dependencies, along with their required versions.
  - Some packages, like `q`, have a `deprecated` field, indicating they are no longer recommended for use.
  - The `engines` field specifies the versions of Node.js, or other runtimes, the package is compatible with.
  - The file is parsed by `npm` or `yarn` during the installation process to ensure consistent and reproducible builds.
    - Example: `node_modules/qs` depends on `side-channel: "^1.1.0"`.
    - Example: `node_modules/finalhandler` depends on `debug: "^4.4.0"`.
    - Example: `node_modules/asn1.js` depends on `bn.js: "^4.0.0"`.
- **WHERE**: The entire structure resides in `package-lock.json` at the root of the

Next Phase →

what improvements I can make in this

Okay, let's analyze potential improvements for this platform, focusing on scalability, reliability, and performance. Based on the provided manifest and code snippets, here's a breakdown with actionable recommendations:

1. Scalability & Performance Bottlenecks:

- **Image Optimization:** The `home.ejs` file uses images directly from external URLs (e.g., Pixabay, static.vecteezy.com, encrypted-tbn0.gstatic.com, cdn-icons-png.flaticon.com).
  - **Bottleneck:** Relying on external CDNs you don't control introduces a single point of failure, latency issues, and potential rate limiting. The page load time is heavily dependent on these external services.
  - **Improvement:**
    1. **Host Images Locally/CDN:** Download these images and serve them from your own CDN (e.g., AWS S3, Cloudflare, or a similar service). This gives you control over caching, availability, and reduces latency.
    2. **Image Optimization:** Optimize images for web delivery (e.g., using tools like ImageOptim, TinyPNG, or libraries like Sharp in Node.js). Reduce file sizes without significant quality loss. Consider using modern image formats like WebP for better compression.
    3. **Lazy Loading:** Implement lazy loading for images, especially in sections like testimonials and feature icons. This improves initial page load time.
- **Lack of Caching:** There's no explicit caching mechanism evident in the provided code. The application likely fetches data from the database on every request.
  - **Bottleneck:** High database load, slow response times, and increased server costs.
  - **Improvement:**
    1. **Implement Caching:** Introduce caching at various levels:
       - **Browser Caching:** Leverage HTTP caching headers (e.g., `Cache-Control`, `Expires`) to instruct browsers to cache static assets (CSS, JavaScript, images). This is partially addressed by using CDN for images but needs explicit configuration.
       - **Server-Side Caching:** Implement server-side caching using Redis or Memcached. Cache frequently accessed data (e.g., listing details, user profiles, category lists, testimonial data). Use a cache invalidation strategy (e.g., time-based expiry, event-based invalidation).

Ask about this phase...

Phase wise Learning + Chatbot Side-by-Side

Phase-Wise Learning Journey

Entering Repo Url, the system creates a full phase wise learning course to explain everything you need for complete understanding.

1. Project Setup & Architecture
2. Core Logic & Data Flow
3. APIs, Features & Mastery

Context-Aware Chatbot

Ask questions at any point. Get precise, code-specific answers without losing context.

1. Understands whole Repo
2. References Exact Code Chunks
3. Full Repository Context (RAG-powered)

+ New Chat

TODAY

HolidayHotels
SYSTEM_DESIGN

HolidayHotels
DEBUGGER

HolidayHotels
ARCHITECT

YESTERDAY

StockTrading-App
ARCHITECT

FinBridgeAA
ARCHITECT

KA kashyap

Hello, kashyap

Here's your repository intelligence overview.

Analyze New Repo

92,331
Total Tokens
Lifetime usage

9
Prompts
Questions asked

$0.0145
Unpaid Balance
Total Due: $0.0145

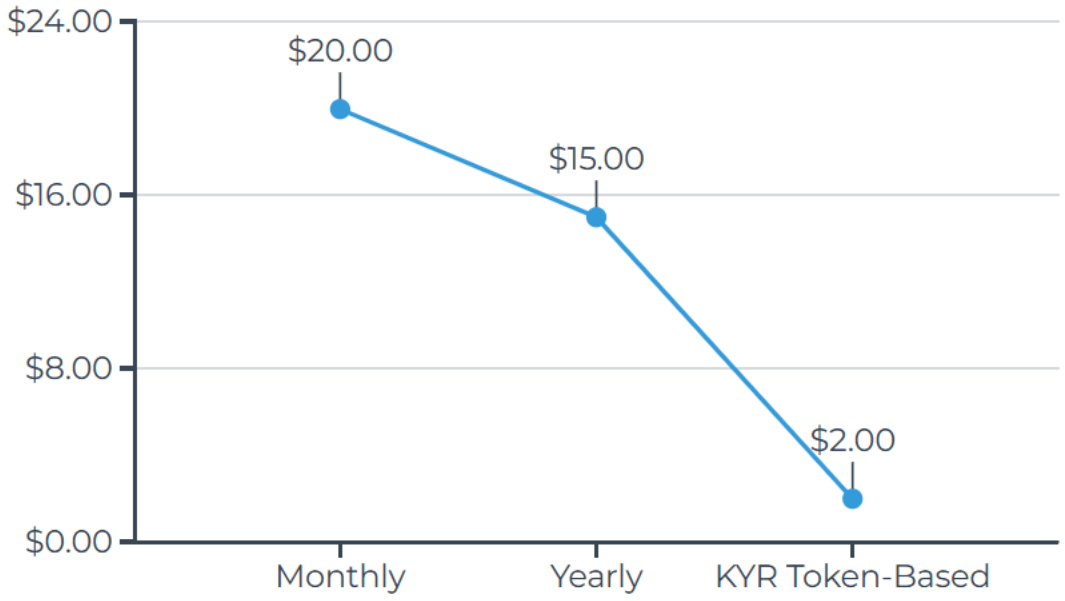
$0
Paid
Lifetime payments

Token Usage (Last 30 Days)

Token-Based Pricing Model

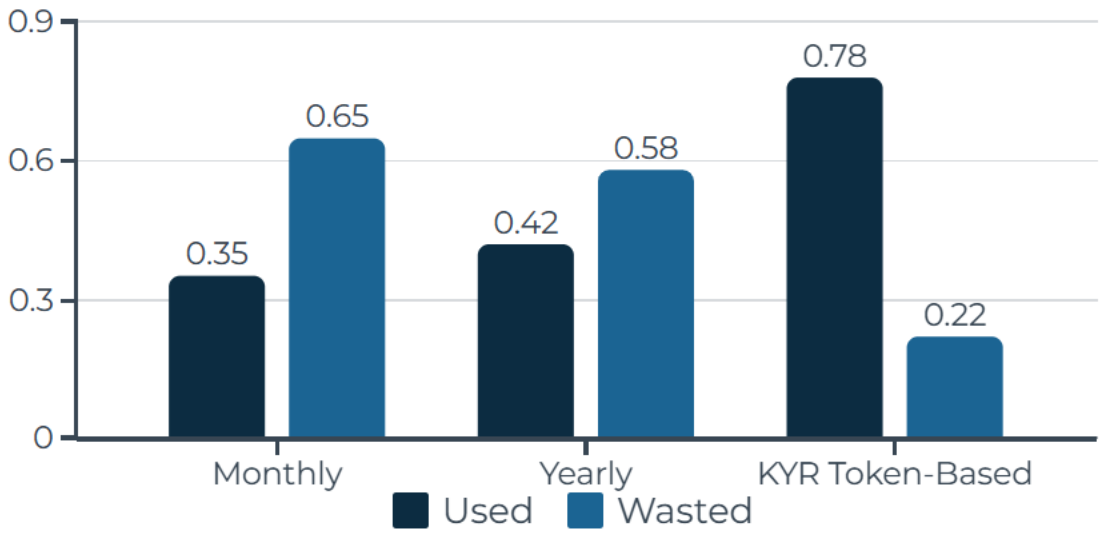
KYR's token-based pricing eliminates the pain points of traditional subscriptions. Instead of paying a fixed monthly or yearly fee regardless of usage, you only pay for the tokens you consume.

Developer Cost Comparison (avg/month)



Beginners save 85–90% cost

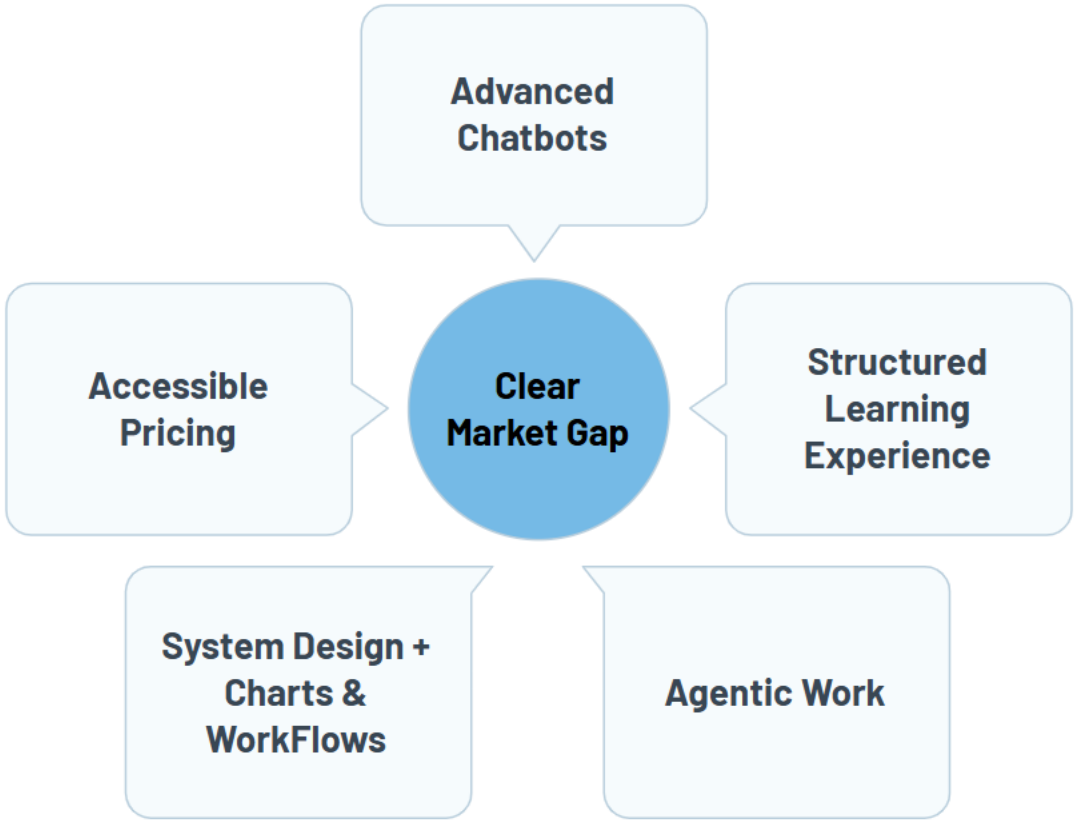
Capacity Utilization



Subscription models force overpayment

KYR's Market Position: Gaps Filled & Growth Opportunity

Gaps in the Current Market that we fill:



KYR's Unique Value Proposition

1. **Structured Learning First**
   Phase-wise understanding vs. random explanations
2. **Full-Repo Intelligence**
   RAG-powered context vs. isolated answers
3. **Accessible to All**
   Token-based pricing vs. expensive subscriptions
4. **4-in-1 Solution**
   Learning, debugging, design, extension in one platform

Repository Ingestion & Chunking: Technical Implementation

KYR utilizes a robust, multi-stage pipeline to transform raw GitHub repositories into semantically searchable knowledge bases, foundational for intelligent developer tools with optimal accuracy and scalability.

1. Repository URL Submission

Users initiate the ingestion pipeline by submitting public GitHub repository URLs through the React frontend.

2. Repository Fetching & Validation

FastAPI validates submitted repository URLs. **PyGithub** then fetches essential files while filtering non-essential assets like binaries and `.git` folders.

3. Semantic Code Chunking

LangChain's RecursiveCharacterTextSplitter parses code and documentation into semantically meaningful **chunks**. These chunks are enriched with metadata such as `file_path`, `chunk_id`, and `file_type` for precise retrieval.

4. Embedding & Vector Storage

Processed chunks are converted into dense **vector embeddings** using sentence-transformers. These are stored in **ChromaDB** with unique namespace isolation and inactive repositories are automatically removed by a background process based on **TTL**.

RAG Retrieval: Context-Aware Response Generation

KYR utilizes a sophisticated Retrieval-Augmented Generation (RAG) pipeline to provide accurate, context-aware responses grounded in actual repository code, eliminating hallucinations.

01

Semantic Similarity Search

User queries are converted into embeddings, then **ChromaDB** performs a vector similarity search to retrieve the most relevant code chunks, enhanced by phase-aware filtering.

02

Context Aggregation & Formatting

Retrieved chunks are aggregated into a structured context document, including metadata like file_path, language, and phase, to preserve code relationships and hierarchy.

03

LLM Processing with Gemini 1.5 Pro

The aggregated context and user question are passed to **Google's Gemini 1.5 Pro** via LangChain. Configured with a low temperature (0.2), the model operates strictly within the provided context, ensuring factual, non-hallucinating responses.

Phase Detection Logic: Intelligent Learning Path Generation

KYR transforms GitHub repositories into structured learning paths through a hybrid approach combining deterministic analysis with intelligent LLM refinement.

Hybrid Phase Detection Strategy

KYR employs a three-layer approach for phase detection: Layers 1 & 2 use deterministic structural analysis and dependency mapping as a foundation, while the LLM is exclusively for naming and explanation without influencing detection logic.

Repository Structure Analysis

The system deterministically identifies entry points and performs folder-based classification to discover module boundaries, revealing the project's natural architecture.

Dependency & Code Flow Analysis

Import statements are extracted to build dependency graphs, revealing module interactions, core logic, and data flow to enable accurate phase sequencing.

Phase Construction & LLM Refinement

Files are assigned to logical phases via rule-based bucketing; the LLM then refines ordering and naming, enhancing user experience while maintaining foundational accuracy.

Thank You!

Thank you for exploring **KYR – Know Your Repo.**

We're transforming how developers understand, debug, scale, and improve any GitHub repository.

Join us in making intelligent code understanding accessible to everyone.

Presented by:

Tech Hustlers

Kashyap B. Rupareliya

Jasmita P. Vekariya